

HYTech Learning Management System

Comprehensive Technical, Security, Scalability, Turnover, and Compliance Audit

Report date: 23 July 2026

System reviewed: HYTech Learning Management System (HYTech LMS)

Repository location: `HYTechLMS/HYTech`

Assessment type: Read-only source, configuration, architecture, build, dependency, and governance review

Document classification: Internal / Confidential

Document purpose

This report consolidates the findings from a read-only review of the HYTech LMS. It assesses whether the system is ready to operate as a complete learning management system and examines security, privacy, academic integrity, feature completeness, mobile and accessibility quality, performance, scalability, developer and staff turnover, operational resilience, and Philippine compliance readiness.

This is an engineering and operational risk assessment, not a formal penetration test, financial audit, certification, or legal opinion. Compliance conclusions require confirmation by the institution's Data Protection Officer (DPO), legal counsel, TESDA compliance personnel, and other responsible officers.

No source code was changed during the assessment. No Git command was executed. A production build was generated outside the repository solely to validate build health and bundle characteristics.

Contents

1. Executive summary
 2. Scope and methodology
 3. Severity model
 4. Critical security and academic-integrity findings
 5. Backend, deployment, and data-lifecycle findings
 6. LMS completeness and correctness
 7. Scalability, performance, and cost
 8. Mobile usability and accessibility
 9. Developer, administrator, and vendor turnover
 10. Philippine privacy and compliance readiness
 11. Operational resilience and enterprise readiness
 12. Dependency and engineering health
 13. Prioritized remediation roadmap
 14. Suggested ownership model
 15. Consolidated finding register
 16. Appendix A — Turnover and handover checklist
 17. Appendix B — Minimum scalability test scenarios
 18. Appendix C — Key external references
 19. Appendix D — Final acceptance statement template
-

1. Executive summary

1.1 Overall conclusion

The HYTech LMS is not ready for production use with real academic or personal data.

The system is not merely unfinished at the user-interface level. Its current backend trust model permits authenticated users to bypass essential LMS controls. Assessment answers can be exposed to students, grades and completion information can be submitted from the browser, class membership is not consistently enforced in Firestore rules, and deactivated users are restricted mainly by frontend routing rather than authoritative backend controls.

The system also has significant privacy exposure, unreliable enrollment Functions, incomplete deployment automation, non-authoritative certificates, inconsistent grading rules, unbounded client-side database reads, no verified backup or recovery workflow, no automated test suite, and weak turnover documentation.

Using the system for official grades, completion decisions, certificates, regulated training records, or sensitive learner information would create material security, privacy, academic-integrity, operational, and reputational risk.

1.2 Overall posture

Domain	Assessment	Summary
Authentication and authorization	Critical	Backend rules do not consistently enforce active status, verified identity, or class membership.
Assessment and grading integrity	Critical	Answer keys are student-readable and grades are calculated and submitted by the client.
Privacy and personal data	Critical	Sensitive and identifying data is overexposed and privacy governance workflows are absent.
Academic records and certificates	Critical	Progress, attempts, and certificates are not sufficiently authoritative or tamper-resistant.
Backend reliability	Critical	Enrollment Functions are stale, structurally inconsistent, and unsafe under transaction or retry behavior.
Feature completeness	High risk	Numerous controls are incomplete, misleading, hardcoded, or local-only.
Scalability and cost	High risk	Whole-collection reads, unbounded listeners, N+1 queries, and large documents increase cost and latency linearly.
Mobile and accessibility	High risk	Responsive behavior exists in places, but touch, keyboard, focus, labeling, and small-screen issues remain.
Developer and staff turnover	Critical	Large monoliths, stale documentation, no automated tests, and incomplete offboarding create severe continuity risk.
Operations and resilience	High risk	No demonstrated staging model, restore drill, monitoring, alerting, incident workflow, or service objectives.
Philippine compliance readiness	Critical	The codebase does not demonstrate the controls needed for DPA accountability or trustworthy regulated training records.

1.3 Most urgent risks

1. Students can read assessment answer keys and submit client-generated scores.
2. Signed-in users can read classes and course content without consistent enrollment checks.
3. Deactivated and unverified users are not authoritatively blocked by backend rules.
4. Progress, quiz attempts, submissions, and activity records can be client-controlled.
5. Personal, educational, and incident information is accessible more broadly than necessary.
6. Privileged test credentials are published and used by a script targeting the named Firebase project.
7. Cloud Functions and the current class/enrollment model are incompatible and unreliable.
8. CI deploys only hosting, allowing frontend and backend authorization/data models to become version-skewed.
9. There is no credible privacy-notice, retention, data-subject-rights, breach-response, or vendor-governance implementation.
10. The system has no automated regression or security-rules test suite.

1.4 Positive observations

The review did identify useful foundations:

- The application builds successfully.
- Firebase is configured in the `asia-southeast1` Firestore location, which is geographically sensible for a primarily Philippine user base.
- Firestore default-deny behavior is retained for unmatched paths.
- Some private profile data has been separated into a restricted subcollection.
- Auto-generated Firestore document IDs are used in several areas.
- Several screens already use role-based routing and real-time listeners.
- A manual acceptance checklist exists, although it is unexecuted and incomplete as a release control.
- The code contains an initial attempt at App Check integration.

These foundations do not offset the critical defects, but they provide a starting point for remediation.

2. Scope and methodology

2.1 Reviewed areas

The review covered:

- React routes, layouts, components, and user workflows.
- Firebase initialization, Authentication use, Firestore access, Storage rules, and App Check initialization.
- Firestore security rules and indexes.
- Cloud Functions source and Functions dependencies.
- Deployment workflow and Firebase project configuration.
- Assessment, assignment, enrollment, progress, notification, incident, user-management, settings, and certificate behavior.
- Hardcoded values, dead controls, stale artifacts, and documentation claims.
- Mobile and accessibility patterns visible from the source.
- Dependency vulnerability audit results.
- Scalability, cost, availability, backup, turnover, and governance readiness.
- Philippine Data Privacy Act and education/TVET compliance risk at a technical-control level.

2.2 Validation performed

- Read-only source and configuration inspection.
- Search for data-access patterns, hardcoded values, unfinished controls, and privacy/security mechanisms.
- Production build directed outside the repository.
- Dependency audit for the frontend and Functions packages.
- Comparison of implementation behavior with Firestore rules and documented application claims.
- Review of current official National Privacy Commission, Firebase, and TESDA guidance.

2.3 Limitations

The review did not include:

- Login to the production Firebase or Google Cloud consoles.
- Inspection of deployed rules, Functions, indexes, backups, billing, IAM, Auth users, logs, or App Check enforcement.
- Active exploitation, destructive testing, or production data access.
- Network penetration testing or external endpoint scanning.
- Interviews with developers, trainers, administrators, learners, the DPO, or management.
- Confirmation of TESDA program registration, institutional licenses, vendor contracts, or policy documents held outside the repository.
- A formal WCAG conformance audit using assistive technology.

Where repository evidence is absent, this report says that a control is **not demonstrated** rather than asserting that the organization has never implemented it elsewhere.

3. Severity model

Severity	Meaning
Critical	Enables material unauthorized access, grade/record manipulation, privacy exposure, or systemic failure; blocks production use.
High	Likely to cause significant operational, compliance, availability, usability, or maintenance harm.
Medium	Important weakness that should be corrected but is not independently a production blocker.
Low	Quality, consistency, or hygiene issue with limited immediate impact.

4. Critical security and academic-integrity findings

SEC-01 — Assessment answer keys are exposed and grades are forgeable

Severity: Critical

Firestore rules permit any authenticated account to read assessment documents:

- `firestore.rules:206-208`

Assessment documents include the full question set and `correctAnswer` values:

- `src/utils/firestoreService.js:3307-3318`

The student browser compares answers with `correctAnswer`, calculates points and percentage, and sends the result back to Firestore:

- `src/components/student/StudentCourse.jsx:50-78`
- `src/components/student/StudentCourse.jsx:1561-1634`

The service accepts client-provided `score`, `earnedPoints`, `passed`, counts, and timing information:

- `src/utils/firestoreService.js:3913-3931`

The rules for attempts verify only limited identity information. They do not authoritatively enforce enrollment, assessment existence, publication state, opening/closing time, attempt count, answer validity, or score bounds:

- `firestore.rules:210-219`

Impact: A student can bypass the UI, retrieve answer keys, submit arbitrary results, attempt assessments repeatedly, and submit outside the allowed schedule. Grades, statistics, completion, and certificates derived from these values cannot be trusted.

Required remediation: Store student-visible questions separately from protected answer keys. Grade on trusted server code. Use deterministic attempt identities or transactional attempt reservations. Enforce enrollment, publication, schedule, attempt count, question version, and score calculations server-side.

SEC-02 — Class and tenant isolation is not consistently enforced

Severity: Critical

Any signed-in account can read all class documents and much of their content:

- Classes and class codes: `firestore.rules:114-147`
- Announcements: `firestore.rules:170-190`
- Assessments: `firestore.rules:206-208`
- Course templates/materials: `firestore.rules:102-110`

There is no consistent backend helper requiring an active enrollment before accessing class content. A class code is therefore not a meaningful access secret because signed-in users can retrieve the class document containing it.

Impact: Students may enumerate classes, retrieve content and answer keys, and interact with classes in which they are not enrolled. This violates least privilege and undermines commercial and academic separation.

Required remediation: Create rule helpers for active user, verified identity, trainer ownership, and active class membership. Separate class discovery/join-code lookup into a narrow, non-sensitive server-mediated workflow.

IAM-01 — Deactivated and unverified users are blocked primarily by frontend code

Severity: Critical

Firestore role helpers check authentication and a role stored in the user's document, but not `status` or verified email:

- `firestore.rules:8-25`

Frontend route components sign users out based on status or verification:

- `src/components/auth/RoleProtectedRoute.jsx:53-79`

However, frontend routing cannot prevent direct Firebase SDK or REST access. The authenticated-route handler also permits a fallback path when profile/status listening fails:

- `src/components/auth/AuthenticatedRoute.jsx:61-65`

Impact: An inactive trainer or administrator can potentially retain backend permissions. An account believed to be disabled may continue reading or changing data using a valid Firebase session.

Required remediation: Disable users through Firebase Admin SDK, revoke refresh tokens, enforce active/verified claims in backend controls, and design a documented offboarding flow that also removes trainer/class assignments.

DATA-01 — Progress, submissions, and activity evidence can be client-controlled

Severity: Critical

- Students can write their own progress documents: `firestore.rules:344-348`.
- Enrollment progress updates do not validate type or range: `firestore.rules:247-252`.
- Progress counts an attempted quiz as completed even when failed:
`src/components/student/StudentCourse.jsx:1057-1080`.
- Submission update rules do not adequately freeze identity, state, and timestamp fields:
`firestore.rules:150-164`.
- Client-created activity logs accept arbitrary action/entity/metadata for the caller: `firestore.rules:331-341` and `src/utils/firestoreService.js:1971-1994`.

Impact: Completion, academic progress, submission state, and audit evidence can be falsified or become internally inconsistent.

Required remediation: Make academic state server-authoritative. Define strict field schemas and immutable identity fields. Derive completion from authoritative events. Separate security/audit logs from user activity feeds.

PRIV-01 — Personal and educational data is overexposed

Severity: Critical

Every signed-in account can read every top-level user profile:

- `firestore.rules:60-64`

Administrative settings can write birth date, phone, address, email, and profile information:

- `src/components/settings/Settings.jsx:278-295`

All signed-in users may read `/userSettings/{uid}` even though settings documents contain profile, privacy, account, security, and notification bundles:

- `firestore.rules:89-94`
- `src/context/useUserSettings.js:78-100`

Enrollment records are broadly readable, and incident-form access is overly broad for trainers:

- `firestore.rules:224-230`
- `firestore.rules:293-305`

Impact: Names, contact information, enrollment status, progress, preferences, incident information, and other learner data may be available to users without a legitimate need.

Required remediation: Split public display profiles from private records, apply field minimization, scope records by role and class, and move sensitive workflows behind server-authorized endpoints.

SEC-03 — Privileged test credentials are published

Severity: Critical

The project README publishes admin, trainer, and student usernames/passwords and describes them as development Firebase accounts:

- `README.md:260-269`

The rules test script contains the same passwords and calls the named Firebase project's REST endpoint rather than a local emulator:

- `scripts/test-rules.ps1:8-9`
- `scripts/test-rules.ps1:48-51`

Impact: If the accounts exist or ever existed in a reachable environment, they should be treated as compromised. The script also creates a risk of tests changing live data.

Required remediation: Rotate/delete the accounts, review authentication and Firestore logs, perform a breach assessment, remove working credentials from documentation, and move all rules tests to the Firebase emulator with isolated fixtures.

SEC-04 — User profiles and identities are insufficiently protected

Severity: High

Users may update their own top-level profile fields as long as `role` and `status` are unchanged:

- `firestore.rules:72-74`

This permits changes to fields such as ID number, UID-like fields, creation metadata, email display, and other identity attributes unless separately constrained.

Administrative email editing updates Firestore but not Firebase Authentication:

- `src/components/users/UserManagement.jsx:471-485`

Impact: Display identity can be manipulated, and the LMS email may diverge from the actual sign-in and password-reset identity.

Required remediation: Define an allow-list of self-editable fields. Move official identity/ID updates to privileged server workflows. Synchronize Auth and profile changes atomically or through a controlled recovery process.

5. Backend, deployment, and data-lifecycle findings

BE-01 — Enrollment Functions are broken and based on a stale data model

Severity: Critical

The application-approval transaction writes an enrollment and then performs another transaction read:

- `functions/src/index.js:40-110`
- Write at `functions/src/index.js:73`
- Later read at `functions/src/index.js:77`

It creates enrollment data containing `courseId` but no `classId`, while current class screens query by `classId`:

- `functions/src/index.js:54-71`
- `src/utils/firestoreService.js:1762-1775`

It updates template-course counters rather than the current class-instance model. The project retains both course applications and class-code enrollment flows.

Impact: Approvals may fail, create invisible enrollments, or corrupt counts. Different screens may disagree about whether a student is enrolled.

Required remediation: Define one canonical model: course template → class instance → class enrollment. Replace stale triggers with idempotent, tested server workflows.

BE-02 — Functions are unsafe under retries and high-volume cleanup

Severity: High

The approval Function generates enrollment IDs with `Date.now()`:

- `functions/src/index.js:55`

An event retry can generate another ID and increment counters again. Cleanup operations load arbitrary numbers of enrollments/applications/materials into a transaction:

- `functions/src/index.js:225-271`
- `functions/src/index.js:286-339`

Impact: Duplicate enrollments, counter drift, transaction failures, partial cleanup, and repeated events are possible.

Required remediation: Use deterministic idempotency keys, event records, bounded pages, queues/bulk writers, checkpoints, retry policies, and reconciliation jobs.

DEP-01 — CI deploys only the frontend

Severity: Critical

The deployment workflow installs dependencies, builds, and deploys only Firebase Hosting:

- `.github/workflows/deploy.yml:27-47`

It does not deploy or validate:

- Firestore rules.
- Firestore indexes.
- Cloud Functions.
- Storage rules.
- App Check enforcement.

It also runs no tests, linting, type checking, dependency gate, or rules-emulator checks; deploys on pushes to `main`; uses `firebase-tools@latest`; and relies on a legacy Firebase token.

Impact: A frontend can be deployed against incompatible rules, indexes, or Functions. Releases are not reproducible or safely gated.

Required remediation: Create environment-gated, version-pinned, full-stack deployments with automated verification, staged rollout, approval, rollback, and workload-identity-based authentication.

DATA-02 — Class and course deletion leaves orphaned records

Severity: High

Class deletion removes only the parent class document:

- `src/utils/firestoreService.js:1091-1101`

Firestore does not automatically delete subcollections. Materials, assessments, attempts, assignments, submissions, announcements, comments, and activity can remain orphaned. Existing cleanup Functions target the stale `courses` model rather than complete class subtrees.

Impact: Privacy retention failures, inconsistent reports, unrecoverable records, unnecessary cost, and inaccessible orphaned content.

Required remediation: Prefer archival for official records. Implement server-side, resumable lifecycle workflows with retention checks, legal holds, deletion manifests, and verification.

DATA-03 — IDs are generated through a race-prone full scan

Severity: High

The service reads the complete users collection, determines the maximum numeric ID, and adds one:

- `src/utils/firestoreService.js:35-50`

Impact: Two concurrent signups can receive the same ID. Read cost grows with every user. Failure falls back to inconsistent values.

Required remediation: Use a transactional allocator, randomized institutional ID, or server-generated identifier with a unique reservation document.

DATA-04 — Routing by class name is ambiguous

Severity: High

Class routing resolves a class by its name and returns the first match:

- `src/components/trainer/ClassDetail.jsx:273-280`

- `src/Utils/firestoreService.js:589-591`

Impact: Duplicate or renamed classes can route users to the wrong record.

Required remediation: Use immutable class document IDs in routes and treat display names as non-unique labels.

6. LMS completeness and correctness

LMS-01 — Grading policy is internally inconsistent

Severity: Critical

The gradebook combines quiz percentages and raw assignment points, then averages them without a consistent weighting model:

- `src/utils/firestoreService.js:4025-4032`
- `src/utils/firestoreService.js:4104-4121`

Passing thresholds disagree:

- Quiz submission uses 60%: `src/components/student/StudentCourse.jsx:1590-1600`.
- Quiz statistics treat 70% as passing: `src/utils/firestoreService.js:4184-4193`.
- Settings claim competency-based grading: `src/components/settings/Settings.jsx:83-84`.

Impact: Learners may receive different pass/fail outcomes depending on the screen or calculation path.

Required remediation: Establish one versioned grading policy supporting weighted categories, competencies, reassessment, manual override with approval, finalization, and a complete change history.

LMS-02 — Certificates are not authoritative

Severity: Critical

Certificate data is manufactured in the browser using enrollment-derived IDs and a default grade of "Pass":

- `src/components/student/StudentCertificates.jsx:145-159`
- `src/components/student/StudentArchivedCourses.jsx:63-85`

The certificate is generated as a client-side image:

- `src/components/student/StudentCertificates.jsx:182-200`

There is no issuance record, authorized approver, signature, verification endpoint, revocation status, certificate version, or evidence bundle.

Impact: Certificates can be fabricated and cannot serve as defensible institutional credentials.

Required remediation: Use server-authorized issuance tied to finalized completion, a unique credential ID, signed/verifiable record, public minimal verification, revocation, reissue, and complete audit history.

LMS-03 — Completion is based on activity rather than mastery

Severity: High

Progress calculations count attempted/submitted items and do not reliably require passing or trainer validation:

- `src/components/student/StudentCourse.jsx:1057-1080`

Impact: A learner may appear complete without demonstrating the required competencies.

Required remediation: Define completion as a server-derived policy over mandatory activities, passing assessments, attendance/training hours, trainer sign-off, and competency evidence.

LMS-04 — Settings advertise controls that are not enforced

Severity: High

Settings include self-registration, admin approval, session timeout, logout, email/SMS, 2FA, password expiry, forced changes, upload limits, and grading options:

- `src/components/settings/Settings.jsx:55-96`
- `src/components/settings/Settings.jsx:560-592`
- `src/components/settings/Settings.jsx:840-887`

Most are saved as preference data but are not consumed by authentication or backend policy. The trainer “Enable 2FA” control has no handler:

- `src/components/trainer/TrainerSettings.jsx:533-543`

Impact: Administrators may believe controls are active when they are not.

Required remediation: Remove or visibly label unavailable controls. Implement security policy on trusted backend systems and add verification tests for every exposed setting.

LMS-05 — Hardcoded and incomplete behavior remains in normal workflows

Severity: High

Examples include:

Issue	Evidence
Every class uses the same Google Meet room	<code>src/components/trainer/ClassDetail.jsx:1945-1974</code>
<code>weeksLeft</code> is fixed at 9	<code>src/components/student/StudentCourse.jsx:997-1001</code>
Download filename references <code>Barista_NC2</code>	<code>src/components/student/StudentCourse.jsx:1694-1697</code>
Download fallback only shows a toast	<code>src/components/student/StudentCourse.jsx:1782-1789</code>
Temporary password is based on surname and birth year	<code>src/components/users/UserManagement.jsx:38-49</code>
Student “View assignment” action only logs to console	<code>src/components/student/StudentCourse.jsx:2287-2298</code>
Student comment delete can show success without deletion	<code>src/components/student/StudentCourse.jsx:3821-3835</code>
Personal calendar events use only local storage	<code>src/components/student/StudentCalendar.jsx:20-24,113-138</code>
Certificates route is disabled/hidden	<code>src/App.jsx:139-141</code>

Impact: Inconsistent behavior, data loss between devices, security weakness, misleading success messages, and loss of user trust.

LMS-06 — Attendance, competency, reporting, and official record workflows are incomplete

Severity: High

The codebase does not demonstrate complete, authoritative workflows for:

- Attendance and training-hour capture.
- Competency evidence and assessor sign-off.
- Reassessment and remediation.
- Grade finalization and appeal.
- Curriculum/version assignment to a class cohort.
- Certificate approval, verification, and revocation.
- Institutional and regulator reporting.
- Guardian or minor-specific workflows.
- Bulk roster import with validation and reconciliation.
- Data corrections with approval and history.

These are essential LMS/TVET capabilities, not optional polish.

7. Scalability, performance, and cost

SCL-01 — Whole-collection reads and unbounded listeners grow linearly

Severity: High

Examples:

- Dashboard reads all users and enrollments: `src/components/dashboard/Dashboard.jsx:89-101`.
- User management subscribes to all users and paginates client-side:
`src/components/users/UserManagement.jsx:135-233`.
- Notifications listen to all documents for a user: `src/utils/firestoreService.js:2070-2086`.
- "Clear all" notifications retrieves all documents: `src/utils/firestoreService.js:2139-2149`.
- Announcement and comment subscriptions commonly have no page boundary.

The source contains approximately 53 `getDocs()` calls and 17 `onSnapshot()` calls, while only a minority of data-access paths use `limit()`.

Impact: Latency, browser memory, mobile data use, and Firestore billing increase with the number of users and historical records.

Required remediation: Add server-filtered queries, cursor pagination, bounded listeners, archival boundaries, TTL where appropriate, and institutional aggregate documents.

SCL-02 — Gradebook and profile loading create N+1 request patterns

Severity: High

Gradebook creation retrieves attempts/submissions for multiple activities and then fetches profiles individually:

- `src/utils/firestoreService.js:4034-4087`

Class roster and activity views also load avatars/profiles separately.

Impact: A class with many students and assessments can generate hundreds or thousands of reads for one screen.

Required remediation: Store bounded display snapshots where appropriate, query in batches, maintain grade summary documents, and move heavy report generation to server-side jobs.

SCL-03 — A global migration runs in every client session

Severity: High

`App.jsx` invokes a "one-time" migration on every app load:

- `src/App.jsx:45-56`

The migration reads every class and performs sequential updates:

- `src/utils/firestoreService.js:4201-4236`

Impact: Repeated global reads, unnecessary cost, startup work, permission errors, and uncontrolled production mutation.

Required remediation: Use versioned administrative migrations with dry-run output, backup, batch boundaries, observability, and rollback planning.

SCL-04 — Large Firestore documents create hard limits and expensive reads

Severity: High

Attachments are stored as base64 in Firestore:

- `src/utils/firestoreService.js:2573-2657`

Assessment questions and answer keys are stored together as arrays in one document:

- `src/utils/firestoreService.js:3305-3327`

`firestore.indexes.json` has no field overrides, so large fields are not explicitly exempted from indexing where appropriate.

Impact: Documents may approach the Firestore 1 MiB limit, unnecessary bytes are downloaded repeatedly, and index/storage cost increases.

Required remediation: Use Cloud Storage for files; normalize large/repeated entities; isolate protected answers; add index exemptions for non-query fields; enforce file and document size server-side.

SCL-05 — Frontend bundle and component size impede performance

Severity: High

The production build succeeded but generated a single JavaScript chunk of approximately:

- **1.58 MB minified**
- **368 KB gzip**

Vite warned that the chunk exceeded 500 KB. All main routes are statically imported in `src/App.jsx`, with no meaningful route-level lazy loading.

Large files include approximately:

- `ClassDetail.jsx` : 6,600 lines.
- `firestoreService.js` : 4,080 lines.
- `StudentCourse.jsx` : 3,700 lines.

Impact: Slow initial loads, more parsing on low-end devices, difficult testing, high regression risk, and poor maintainability.

Required remediation: Add route-level lazy loading, split feature modules, remove dead dependencies/assets, and enforce bundle budgets in CI.

SCL-06 — No capacity model, load testing, or cost governance is demonstrated

Severity: High

There is no defined or tested capacity for:

- Concurrent logins.
- Simultaneous assessment submissions.
- Students per class.
- Classes per trainer.
- Historical notifications/comments/attempts.
- Bulk enrollment, graduation, or archival.
- Read/write cost per active learner.

Required remediation: Establish load scenarios, service objectives, budget alerts, query dashboards, and a capacity review before every major enrollment increase.

8. Mobile usability and accessibility

UX-01 — Layout assumptions remain unsafe for mobile browsers

Severity: High

Role layouts use combinations of `h-screen`, `w-screen`, fixed maximum heights, and overflow clipping:

- `src/components/student/StudentDashboardLayout.jsx:130-143`

These patterns are fragile with mobile browser address bars, virtual keyboards, orientation changes, and screens taller than the fixed maximum.

Some fixed notifications use a minimum width near the width of a 320 px device:

- `src/components/student/StudentCourse.jsx:1839`

UX-02 — Hover-only controls are inaccessible on touch and keyboard

Severity: High

Actions are sometimes hidden using `opacity-0 group-hover:`

- Student course activity action: `src/components/student/StudentCourse.jsx:2288`.
- Trainer task action: `src/components/trainer/Tasks.jsx:193`.

Touch users may never discover these actions, while keyboard visibility and focus styling are inconsistent.

UX-03 — Form labeling and control semantics are incomplete

Severity: High

Many visible labels are not connected to inputs with `htmlFor` / `id`. Numerous icon buttons lack an accessible name. Mobile menu buttons lack complete `aria-label`, `aria-expanded`, and `aria-controls` state:

- `src/components/student/StudentSidebar.jsx:167-175`
- `src/components/layout/AdminSidebar.jsx:66-74`

Custom modals do not consistently demonstrate focus trapping, Escape handling, initial focus, or focus restoration.

Required remediation: Adopt WCAG 2.2 AA as the engineering target; implement semantic components; test keyboard-only, screen reader, 200% zoom, reduced motion, color contrast, and common mobile widths.

9. Developer, administrator, and vendor turnover

TOV-01 — Extreme bus-factor concentration

Severity: Critical

Essential behavior is concentrated in a few very large files. Enrollment, assessment, notifications, files, progress, comments, and grading share one service module, while trainer and student workflows are monolithic components.

Impact: A departing developer takes undocumented knowledge with them. New developers cannot safely isolate changes and are more likely to introduce cross-feature regressions.

Required remediation: Decompose by bounded domain, define service interfaces and schemas, create maintainers/owners, and require tests before extraction or rewrite.

TOV-02 — Documentation is stale and contradicts the implementation

Severity: Critical

The README schema documents a `supervisor` role and data shapes that do not match the current class/subcollection implementation:

- `README.md:437-500`

The README also claims production readiness, responsive design, accessibility, and code splitting that are not demonstrated by the implementation.

Impact: A replacement developer may build against the wrong schema or assume controls exist when they do not.

Required remediation: Replace feature-marketing documentation with an authoritative architecture guide, live data dictionary, permissions matrix, and known-limitations register.

TOV-03 — No automated regression safety net

Severity: Critical

The frontend scripts include only `dev`, `build`, and `preview`:

- `package.json:19-23`

No repository test suite was identified. The acceptance checklist is manual and appears unsigned/unexecuted:

- `TESTING_CHECKLIST.md:1-353`

Required remediation: Add Firestore emulator rules tests, unit tests, component tests, Playwright role journeys, accessibility checks, dependency scanning, linting, formatting, type checking, and release gates.

TOV-04 — Environment and production ownership are insufficiently separated

Severity: High

`.firebaseerc` defines one default Firebase project:

- `.firebaseerc:1-5`

There is no documented development/staging/production topology, anonymized test dataset, or safe migration environment.

Impact: New staff can accidentally test, seed, migrate, or deploy against production.

Required remediation: Separate cloud projects and IAM groups; use least-privilege service identities; provide `.env.example`; prohibit personal owner accounts; document access approval and recovery.

TOV-05 — Staff offboarding does not remove real authority

Severity: Critical

Deactivation updates a profile status but does not disable Firebase Auth, revoke tokens, remove co-trainer access, transfer ownership, or invalidate active sessions.

Required remediation: Create an offboarding workflow covering Auth disablement, token revocation, class transfer, group/role removal, API/integration access, active session review, device recovery, and audit evidence.

TOV-06 — No vendor exit or tested data portability strategy

Severity: High

The system is closely coupled to Firestore document paths and client SDK behavior. No complete export/import, schema versioning, restoration, or cloud-exit process is demonstrated.

Required remediation: Define an institution-owned canonical export format, scheduled exports, restoration testing, media export, integrity hashes, dependency inventory, and a documented Firebase/vendor exit plan.

10. Philippine privacy and compliance readiness

10.1 Applicable baseline

Assuming the institution operates in the Philippines, the primary privacy baseline is Republic Act No. 10173, the Data Privacy Act of 2012, its Implementing Rules and Regulations, and National Privacy Commission issuances.

Education records are sensitive personal information under the NPC's definition. The LMS processes identity, birth date, contact data, enrollment, grades, assessment responses, progress, incident reports, and certificate information. This requires a higher level of organizational, physical, and technical control.

CMP-01 — No adequate privacy notice or processing explanation at registration

Severity: Critical

Signup collects full name, birth date, email, and mobile number:

- `src/components/auth/SignUp.jsx:383-525`

No user-facing privacy notice, DPO contact, processing-purpose summary, recipient/vendor disclosure, retention information, rights explanation, or terms acceptance is presented.

The DPA requires transparency, legitimate purpose, proportionality, and information about processing and rights. Consent is not necessarily the correct basis for every educational record; the institution must document the applicable lawful basis per purpose.

Required remediation: Create layered, plain-language privacy notices reviewed by the DPO/counsel. Maintain a processing inventory mapping each field and purpose to lawful basis, retention, recipients, access roles, and disposal.

CMP-02 — Signup drafts persist personal data in local storage

Severity: High

The form stores name, birth date, email, and phone in browser `localStorage`:

- `src/components/auth/SignUp.jsx:17-90`

The "clear on load" effect is followed by another effect that writes the state back.

Impact: Personal data can remain accessible on a shared device.

Required remediation: Do not persist sensitive registration drafts by default. If draft recovery is genuinely needed, use explicit consent, short expiry, minimal fields, secure server association, and a visible clear mechanism.

CMP-03 — Data minimization and least privilege are not met

Severity: Critical

The broad reads described in PRIV-01 conflict with the DPA principles of transparency, legitimate purpose, and proportionality. User and educational data is accessible beyond the minimum role/class need.

Required remediation: Perform a field-level access review and redesign the database around data classification and purpose-limited access.

CMP-04 — Data-subject rights are not operationalized

Severity: High

No complete workflow exists for access, source/recipient disclosure, correction across duplicate records, objection, withdrawal where applicable, erasure/blocking, portability, or complaint escalation.

Required remediation: Establish authenticated request workflows, identity verification, case tracking, deadlines, approvals, export format, exception handling, and evidence of fulfillment.

CMP-05 — Retention, archival, disposal, and legal hold are undefined

Severity: Critical

Some records accumulate indefinitely, some deletions leave orphans, and administrators can export and purge activity logs to a workstation CSV:

- `src/components/logs/SystemLogs.jsx:305-325`
- `src/utils/firestoreService.js:2282-2295`

Required remediation: Approve retention categories for applicant data, user profiles, academic records, attempts, submissions, incidents, certificates, communications, notifications, and security logs. Implement archive, legal hold, secure disposal, and deletion verification.

CMP-06 — Audit and breach-response evidence is inadequate

Severity: Critical

Activity logs are partially client-created and can be purged. No centralized alerting, breach case management, evidence preservation, or notification workflow is demonstrated.

Qualifying Philippine personal-data breaches may require notification to the NPC and affected data subjects within 72 hours of knowledge or reasonable belief. The institution cannot reliably meet that obligation without monitoring, ownership, triage, communication templates, and rehearsed procedures.

Required remediation: Establish a security incident response plan, trusted server logs, alerting, event classification, DPO escalation, evidence preservation, breach assessment, notification templates, and annual exercises.

CMP-07 — DPO, registration, PIA, and annual reporting controls require verification

Severity: High

The repository does not demonstrate:

- An appointed and reachable DPO.
- A privacy management program/manual.
- A Privacy Impact Assessment for the LMS.
- Registration of the DPO/data processing system where required.
- Annual Security Incident Report ownership.

- Employee privacy training.
- Vendor privacy review.

These may exist outside the repository and must be verified organizationally.

CMP-08 — Firebase, AI, and other vendors require processor governance

Severity: High

The chatbot can use a client Gemini key or send messages and a client-supplied role to a configured proxy:

- `src/components/hytbot/HytBot.jsx:8-17`
- `src/components/hytbot/HytBot.jsx:310-340`

No Firebase authentication token is shown in the proxy call. Chat messages may contain learner or institutional information.

Required remediation: Authenticate the proxy, enforce rate limits and role server-side, minimize and redact data, define AI retention/training rules, publish clear notice, and execute appropriate vendor processing/security agreements. The institution remains accountable for transferred data.

CMP-09 — Minor and guardian safeguards are absent

Severity: High

Signup accepts any birth date up to the current date:

- `src/components/auth/SignUp.jsx:462-475`

No minimum-age policy, guardian process, age-appropriate notice, recording/webcam rules, or minor-specific incident handling is shown.

Required remediation: Determine the learner population and applicable guardian requirements. Implement age-appropriate notices and controls with DPO/legal review.

CMP-10 — Accessibility compliance is not demonstrated

Severity: High

The accessibility issues in Section 8 contradict documentation claims. The exact legal requirement depends on institutional status and contracts, but WCAG 2.2 AA should be the engineering acceptance standard.

CMP-11 — TESDA/TVET compliance is unproven

Severity: Critical if used for official TVET records

The repository includes a hardcoded TESDA-like catalog, but catalog names do not establish program registration or compliance. If the LMS supports TESDA-registered programs, it must map current registered programs, qualifications, competency standards, trainers, learning evidence, attendance/training hours, learner record books, assessment outcomes, and required reports.

Current client-controlled grades, approximate progress, and non-authoritative certificates are unsuitable as official evidence.

Required remediation: Conduct a joint requirements mapping with TESDA compliance personnel. Treat the LMS as non-authoritative until every required record, approval, retention, correction, and report has a tested control.

11. Operational resilience and enterprise readiness

OPS-01 — No verified backup, restore, or disaster-recovery process

Severity: Critical

The repository contains no evidence of scheduled Firestore backups, point-in-time recovery, Storage backup, recovery objectives, restoration scripts, or restore exercises.

Required remediation: Define RPO/RTO values, enable appropriate backups/PITR, include media and configuration, document recovery, and perform periodic restoration drills into an isolated project.

OPS-02 — Monitoring and alerting are insufficient

Severity: High

Error handling is dominated by browser `console.error` / `console.warn`. No centralized application monitoring, structured logs, trace/correlation IDs, uptime checks, function failure alerts, security alerts, or data-quality reconciliation dashboard is demonstrated.

Required remediation: Add client and server error monitoring, structured audit/security logs, cloud alerts, synthetic role journeys, data integrity checks, and operational dashboards.

OPS-03 — No service objectives or support model

Severity: High

No uptime objective, support hours, incident severity definitions, escalation matrix, maintenance window, response target, or learner communication process is documented.

OPS-04 — Hosting security hardening is incomplete

Severity: High

`firebase.json` contains no Content Security Policy, Referrer Policy, Permissions Policy, HSTS, anti-framing policy, or other explicit hosting security headers:

- `firebase.json:1-27`

App Check initialization is optional and CI does not provide the production key:

- `src/firebase.js:39-52`

Required remediation: Define tested security headers, enforce App Check where suitable, restrict API keys by API/referrer as appropriate, and add server-side abuse/rate controls.

OPS-05 — CSV exports permit spreadsheet formula injection

Severity: High

CSV values are quoted but not neutralized when beginning with `=`, `+`, `-`, or `@`:

- `src/components/logs/SystemLogs.jsx:60-71`
- `src/components/trainer/ClassDetail.jsx:1925-1942`

Impact: Opening exported data in spreadsheet software can evaluate a malicious learner-controlled value as a formula.

Required remediation: Prefix dangerous cells, use a safe export library, restrict formulas, and warn administrators about untrusted exports.

12. Dependency and engineering health

12.1 Dependency audit results

The dependency scanner reported:

Package area	Total	Critical	High	Moderate	Low
Frontend	11	2	3	5	1
Cloud Functions	10	1	5	4	0

Reported dependency families include `protobufjs`, `websocket-driver`, Vite, gRPC, React Router, PostCSS, and `firebase-admin`.

Scanner results require dependency-tree and exploitability triage; they do not prove that every advisory is reachable. Nevertheless, critical/high findings and old Functions dependencies are unacceptable without documented resolution or risk acceptance.

12.2 Runtime and maintenance concerns

The Functions package specifies Node 18 and older Firebase Admin/Functions generations:

- `functions/package.json:6-19`

There is no automated dependency update service, software bill of materials, license scan, or security release SLA.

13. Prioritized remediation roadmap

Phase 0 — Containment: immediately to 7 days

1. **Stop production-readiness and compliance claims.** Clearly label the system as development/pilot only.
2. **Rotate/delete published accounts.** Review Auth, Firestore, Hosting, and Function logs and perform a DPO-led incident assessment.
3. **Freeze official grading and certificate use.** Do not use current results for formal decisions.
4. **Restrict Firestore access.** Introduce active-user and class-membership controls; separate private profiles/settings.
5. **Protect answer keys.** Remove them from student-readable documents and disable client-authoritative scoring.
6. **Disable inactive accounts in Firebase Auth** and revoke tokens.
7. **Preserve evidence.** Pause ad hoc audit-log purging and define interim retention.
8. **Create separate development/staging/production cloud projects.** Prevent tests and migrations from reaching production.

Phase 1 — Trust foundation: 8 to 30 days

1. Define the canonical course-template, class, enrollment, assessment, attempt, grade, completion, and certificate models.
2. Implement server-authoritative enrollment and grading workflows.
3. Rewrite Firestore rules with emulator tests for every role and attack case.
4. Move files to Cloud Storage with membership-aware rules and server validation.
5. Implement trusted security/audit logs separate from user activity feeds.
6. Add full-stack CI: lint, type check, unit tests, rules tests, build, dependency gate, end-to-end smoke tests, staged deployment.
7. Create privacy notice, data inventory, PIA, retention schedule, and data-subject request process with the DPO.
8. Establish monitoring, alerting, incident response, and backup configuration.

Phase 2 — Complete the LMS: 31 to 90 days

1. Implement the approved grading/competency policy.
2. Add attendance/training-hour and competency evidence workflows.
3. Implement authoritative completion, certificate issuance, verification, revocation, and reissue.
4. Add grade finalization, appeals, corrections, and audit history.
5. Add pagination, aggregates, bounded listeners, queues, and reconciliation jobs.
6. Remove hardcoded/dead controls and make feature status explicit.
7. Conduct mobile, browser, keyboard, screen-reader, contrast, and zoom testing.
8. Produce current architecture, schema, operations, recovery, and turnover documentation.

Phase 3 — Production qualification

The system should not be approved for production until all of the following are independently demonstrated:

- No open Critical findings.
 - High findings have been remediated or formally accepted by accountable management.
 - Firestore rules pass deny/allow tests for all roles and cross-class attacks.
 - Server-side grading cannot be altered by the client.
 - Load tests meet approved SLOs and cost expectations.
 - Backup restoration succeeds within approved RPO/RTO.
 - DPO signs off the PIA, notices, retention, rights, vendor, and breach controls.
 - TESDA/regulatory requirements are mapped and accepted where applicable.
 - WCAG acceptance testing is completed.
 - End-to-end role journeys pass in staging.
 - Deployment and rollback are rehearsed.
 - Administrator and trainer offboarding is tested.
-

14. Suggested ownership model

Workstream	Accountable owner	Required participants
Security rules and IAM	Technical lead / Security owner	Firestore engineer, QA, DPO
Assessment and grading	Academic director	Trainers, registrar, developer, QA
Privacy compliance	DPO	Legal counsel, technical lead, records officer
TESDA mapping	Compliance/registrar	Academic director, trainers, DPO, developer
Infrastructure and deployment	Platform owner	Developer, security owner, QA
Backup and disaster recovery	Platform owner	Management, records officer, DPO
Accessibility	Product owner	Learners with disabilities, QA, frontend developer
Turnover and documentation	Engineering manager	All system owners
Incident response	Incident commander / DPO	Security, legal, management, communications

No critical production capability should be owned only by one individual.

15. Consolidated finding register

ID	Severity	Finding	Production blocker
SEC-01	Critical	Answer keys exposed and grades forgeable	Yes
SEC-02	Critical	Missing class-membership isolation	Yes
IAM-01	Critical	Inactive/unverified users not blocked authoritatively	Yes
DATA-01	Critical	Client-controlled academic and activity records	Yes
PRIV-01	Critical	Personal/educational data overexposure	Yes
SEC-03	Critical	Published privileged credentials	Yes
SEC-04	High	Insufficient user-field and Auth/profile identity controls	Yes
BE-01	Critical	Broken/stale enrollment Functions	Yes
BE-02	High	Non-idempotent retries and unbounded cleanup	Yes
DEP-01	Critical	Hosting-only deployment and missing release gates	Yes
DATA-02	High	Orphaned records on deletion	Yes
DATA-03	High	Race-prone ID allocation	No
DATA-04	High	Ambiguous name-based routing	No
LMS-01	Critical	Inconsistent grading policy	Yes
LMS-02	Critical	Non-authoritative certificates	Yes
LMS-03	High	Completion not based on mastery	Yes
LMS-04	High	Security/settings controls not enforced	Yes
LMS-05	High	Hardcoded and incomplete workflows	No
LMS-06	High	Missing core academic/TVET workflows	Yes
SCL-01	High	Unbounded reads/listeners	No
SCL-02	High	N+1 gradebook/profile queries	No
SCL-03	High	Client-run global migration	Yes
SCL-04	High	Large Firestore documents/base64 files	No
SCL-05	High	Large single bundle and monoliths	No
SCL-06	High	No capacity/load/cost model	Yes
UX-01	High	Unsafe mobile layout assumptions	No
UX-02	High	Hover-only controls	No
UX-03	High	Incomplete accessibility semantics	No
TOV-01	Critical	Extreme bus factor	Yes

ID	Severity	Finding	Production blocker
TOV-02	Critical	Stale/contradictory documentation	Yes
TOV-03	Critical	No automated regression suite	Yes
TOV-04	High	Weak environment separation	Yes
TOV-05	Critical	Unsafe staff offboarding	Yes
TOV-06	High	No vendor exit/data portability plan	No
CMP-01	Critical	No adequate registration privacy notice	Yes
CMP-02	High	PII stored in local signup draft	No
CMP-03	Critical	Minimization/least privilege deficiencies	Yes
CMP-04	High	No operational data-subject rights workflow	Yes
CMP-05	Critical	Undefined retention/disposal/legal hold	Yes
CMP-06	Critical	Inadequate audit and breach response	Yes
CMP-07	High	DPO/PIA/registration/reporting not demonstrated	Yes
CMP-08	High	Vendor/AI processing governance gaps	Yes
CMP-09	High	Minor/guardian safeguards absent	Conditional
CMP-10	High	Accessibility compliance not demonstrated	Yes
CMP-11	Critical	TESDA/TVET compliance unproven	Conditional
OPS-01	Critical	No verified backup/restore/DR	Yes
OPS-02	High	Insufficient monitoring and alerting	Yes
OPS-03	High	No SLO/support model	No
OPS-04	High	Missing hosting/App Check hardening	Yes
OPS-05	High	CSV formula injection	No

Appendix A — Turnover and handover checklist

Before responsibility is transferred to a new developer, administrator, or vendor, the institution should possess:

- System context, container, component, and data-flow diagrams.
 - Current collection and document schema with field types and classification.
 - Role/permission matrix including backend rule tests.
 - Environment inventory and cloud project ownership.
 - Deployment, rollback, migration, backup, and restore runbooks.
 - Dependency and software bill of materials inventory.
 - Secrets/API key inventory and rotation schedule.
 - Vendor contracts, processor roles, data locations, and exit procedures.
 - Current finding/risk register and accepted exceptions.
 - Support and incident escalation matrix.
 - Named owners for grading, certificates, privacy, records, infrastructure, and security.
 - Test accounts and test data that exist only in non-production environments.
 - Automated acceptance suite and last successful release evidence.
 - Staff offboarding and class-ownership transfer workflow.
 - Data retention, legal hold, archive, and disposal procedures.
-

Appendix B — Minimum scalability test scenarios

1. Simultaneous login and dashboard load for expected peak active users.
2. Full class opening with the maximum roster, modules, materials, announcements, and assessments.
3. Simultaneous quiz start and submission at class and institution peak.
4. Trainer gradebook loading for the largest expected class and assessment count.
5. Bulk class enrollment, transfer, completion, and archival.
6. Notification fan-out to all learners and subsequent read/clear behavior.
7. Large permitted file upload/download over slow mobile connections.
8. Function retry, duplicate event, partial failure, and reconciliation.
9. Backup restoration into an isolated project.
10. Rules/index deployment rollback.
11. Loss of a trainer/admin account during an active class.
12. Data-subject export and erasure request across all collections and media.

Each scenario should record latency percentiles, error rate, document reads/writes, function duration/retries, client memory, bundle/load timings, and estimated cost.

Appendix C — Key external references

- National Privacy Commission, **Data Privacy Act of 2012**: <https://privacy.gov.ph/data-privacy-act/>
 - National Privacy Commission, **Implementing Rules and Regulations**: <https://privacy.gov.ph/implementing-rules-regulations-data-privacy-act-2012/>
 - National Privacy Commission, **Data Subject Rights**: <https://privacy.gov.ph/data-subject-rights/>
 - National Privacy Commission, **Retention and Disposal Guidance**: <https://privacy.gov.ph/day-to-day/>
 - National Privacy Commission, **Appointing a Data Protection Officer**: <https://privacy.gov.ph/appointing-a-data-protection-officer/>
 - National Privacy Commission, **Breach Reporting Procedures**: <https://privacy.gov.ph/exercising-breach-reporting-procedures/>
 - National Privacy Commission, **Education Sector Advisory No. 2020-1**: <https://privacy.gov.ph/wp-content/uploads/2020/10/DP-Council-Education-Sector-Advisory-No.-2020-1.pdf>
 - Firebase, **Cloud Firestore Best Practices**: <https://firebase.google.com/docs/firestore/best-practices>
 - Firebase, **Cloud Firestore Usage and Limits**: <https://firebase.google.com/docs/firestore/quotas>
 - Firebase, **Cloud Firestore Pricing and Billing**: <https://firebase.google.com/docs/firestore/pricing>
 - Firebase, **Aggregation Guidance**: <https://firebase.google.com/docs/firestore/solutions/aggregation>
 - TESDA, **Program Registration and Accreditation**: <https://tesda.gov.ph/About/TESDA/26>
-

Appendix D — Final acceptance statement template

The following should be signed only after evidence is reviewed:

Management acknowledges that the HYTech LMS has completed security, privacy, academic-integrity, accessibility, scalability, disaster-recovery, and regulatory acceptance testing. Critical findings are closed. Residual risks are documented with accountable owners and expiry dates. The DPO has approved the privacy controls, and academic/TESDA owners have approved the authoritative record workflows applicable to the institution.

Role	Name	Signature	Date
Executive sponsor			
Technical owner			
Security owner			
Data Protection Officer			
Academic/registrar owner			
TESDA compliance owner, if applicable			
QA/release approver			

End of report